# Mise en place d’un serveur Apache avec un certificat autosigné TLS

## Sommaire

**Sommaire**.....1
**Informations additionnelles**.....1
**Partie 1 - Présentation de TLS**.....2
**Partie 2 - Les prérequis**.....2
**Partie 3 - Installation et configuration d’Apache2**.....2
**Partie 4 : Génération d’un certificat auto signé avec OpenSSL**.....4
**Partie 5 : Configuration d’un VirtualHost HTTPS**.....5
**Partie 6 : Tests et analyse des logs**.....7
**Partie 7 : Questions (Annexe)**.....8

---

## Informations additionnelles

| | |
|---|---|
| Os utilisé | Debian 13.1 |
| Procédure réalisée sur machine virtuelle : | Oui |

# Partie 1 - Présentation de TLS

TLS (**Transport Layer Security**) est un protocole de sécurité qui permet de chiffrer les échanges entre un client (comme un navigateur) et un serveur, afin de garantir la confidentialité, l'intégrité des données et l'authentification lors des connexions HTTPS.

---

# Partie 2 - Les prérequis

1) Une machine sous Debian avec une connexion à internet
2) Un navigateur web pour réaliser les tests

---

# Partie 3 - Installation et configuration d'Apache2

Nous allons installer apache2 qui est le service nécessaire afin de monter un serveur web.

```
apt update
apt install apache2 -y
```

On peut vérifier que le serveur ai bien démarré à l'aide de la commande suivante :

```
systemctl status apache2
```

On peut également se connecter via l'ip de la machine, dans un navigateur web. Cette page devrait s'afficher.

Nous allons activer le démarrage automatique du service au lancement de la machine à l'aide de la commande suivante :

```
systemctl enable apache2
```

Il est bon de savoir que le répertoire racine par défaut sous Debian, se situe ici :

```
var/www/html
```

Nous allons donc modifier le fichier index.html et y ajouter ceci afin d'avoir une page plus présentable.

```
<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width,
```

```
initial-scale=1.0" />
    <title>Accueil</title>
    <style>
        body {
            margin: 0;
            font-family: Arial, sans-serif;
            background: #f4f4f4;
            color: #333;
        }
        header {
            background: #1e88e5;
            color: white;
            padding: 20px;
            text-align: center;
        }
        nav {
            background: #1565c0;
            padding: 10px;
            display: flex;
            justify-content: center;
            gap: 20px;
        }
        nav a {
            color: white;
            text-decoration: none;
            font-size: 18px;
        }
        .hero {
            padding: 100px 20px;
            text-align: center;
            background: white;
        }
        .hero h1 {
            margin-bottom: 20px;
        }
        footer {
            text-align: center;
            padding: 20px;
            background: #333;
            color: #fff;
            margin-top: 40px;
        }
    </style>
</head>
<body>

<header>
    <h1>Bienvenue sur mon site</h1>
</header>

<nav>
    <a href="#">Accueil</a>
    <a href="#">À propos</a>
```

```
    <a href="#">Services</a>
    <a href="#">Contact</a>
</nav>

<section class="hero">
    <h1>Page d'accueil</h1>
    <p>Bienvenue sur votre nouvelle page d'accueil ! Personnalisez-la
comme vous le souhaitez.</p>
</section>

<footer>
    (c) 2025 - Mon Site Web
</footer>

</body>
</html>
```

Si on rafraîchit la page dans le navigateur, le changement doit apparaître.

---

# Partie 4 : Génération d'un certificat auto signé avec OpenSSL

Nous allons générer

- Une clé privée (server.key)
- Un certificat auto signé (server.crt) qui ne sera valide que 365 jours.

Nous allons créer le dossier "ssl" car par défaut sous Debian, il n'est pas créé.

```
cd /etc/apache2
mkdir ssl
cd ssl
```

Nous allons donc générer la clé rsa privé de 365 jours

```
openssl req -x509 -nodes -days 365 -newkey rsa:2048 -keyout
/etc/apache2/ssl/server.key -out /etc/apache2/ssl/server.crt
```

Il faudra ensuite les informations demandés, voici un exemple :

```
FR (Pays)
Calvados (Département)
Caen (Ville)
Armel (Nom entreprise)
Armel (Nom entreprise)
172.17.23.90 (ip du serveur)
```

```
armel.plantier@protonmail.com (mail)
```

La raison pour laquelle on utilise l'option -nodes dans cette commande OpenSSL est **spécifiquement pour éviter de chiffrer la clé privée avec un mot de passe**.

Si on liste les fichier présent dans le répertoire "ssl" nous devons retrouver "server.key" et "server.crt"

```
ls
```

```
root@debian:/etc/apache2/ssl# ls
server.crt  server.key
```

---

# Partie 5 : Configuration d'un VirtualHost HTTPS

Nous allons commencer par activer le module SSL d'apache

```
a2enmod ssl
systemctl restart apache2
```

Une fois le module ssl activé, nous allons devoir mettre en place le site nommé "site-secure.conf"

```
nano /etc/apache2/sites-available/site-secure.conf
```

Il faudra coller ceci en y entrant l'@IP de votre machine :

```
<IfModule mod_ssl.c>
<VirtualHost *:443>
ServerName 172.17.23.90
DocumentRoot /var/www/html
SSLEngine on
SSLCertificateFile /etc/apache2/ssl/server.crt
SSLCertificateKeyFile /etc/apache2/ssl/server.key
ErrorLog ${APACHE_LOG_DIR}/error.log
CustomLog ${APACHE_LOG_DIR}/access.log combined
</VirtualHost>
</IfModule>
```

→ "SSLEngine on" est une directive Apache qui permet d'activer le chiffrement SSL dans un virtualhost.

→ Afin d'activer le VirtualHost nous allons utiliser la commande suivante :

```
a2ensite nom_du_fichier.conf
```

Donc

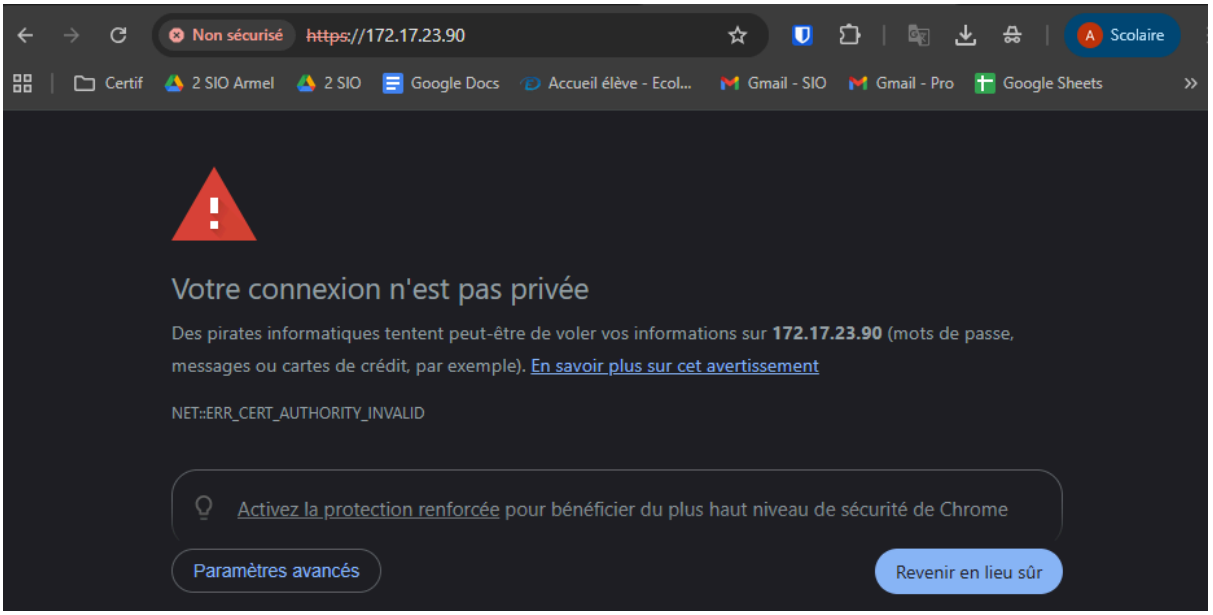
```
a2ensite site-secure.conf
```

Il faudra ensuite recharger apache2

```
systemctl reload apache2
```

→ Nous allons pouvoir tester la connexion en HTTPS au site en entrant l'IP dans le navigateur.

```
https://172.17.23.90/
```




Si tout se passe bien vous aurez cet avertissement de sécurité. Cela se produit car notre certificat n'est pas reconnu. On peut même remarquer que la durée du certificat est bien de 1 an.

Si on poursuit, on remarquera bien que la connexion HTTPS est bien établie.

# Partie 6 : Tests et analyse des logs

→ Afin de vérifier la configuration d'apache on peut faire la commande suivante :

```
apachectl configtest
```

Si nous n'avons aucune erreur, "Syntax OK" doit nous être retourné. En cas d'erreur, elle nous est parfois donné dans le message d'erreur, par exemple celui ci :

```
Syntax error on line 45 of
/etc/apache2/sites-available/000-default.conf:
Invalid command 'SometingWrong', perhaps misspelled or defined by a
module not included in the server configuration
```

Cela nous indique à quel point se trouve l'erreur. Si cela n'aide pas, on peut tenter de redémarrer le service apache2.

Conseil pratique :

- Si l'erreur n'est pas évidente, commentez la ligne problématique (#) et testez à nouveau.
- Apache refuse de démarrer si la syntaxe est incorrecte, donc toujours tester avant de redémarrer.

→ Pour nos tests, les chemin des fichiers de logs sont les suivants :

```
Logs d'accès : /var/log/apache2/access.log
Logs d'erreurs : /var/log/apache2/error.log
```

Si on veut afficher les 10 premières lignes on fait :

```
tail -n 10 /var/log/apache2/access.log
tail -n 10 /var/log/apache2/error.log
```

Voici ce que doit nous donner cette réponse.

```
root@debian-12:~# tail -n 10 /var/log/apache2/access.log
172.17.12.18 - - [27/Nov/2025:14:29:38 +0100] "GET /favicon.ico HTTP/1.1" 404 490 "http://172.17.23.90/" "Mozilla/5.
 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/142.0.0.0 Safari/537.36"
172.17.12.18 - - [27/Nov/2025:14:30:19 +0100] "-" 408 0 "-" "-"
172.17.12.18 - - [27/Nov/2025:14:32:39 +0100] "GET / HTTP/1.1" 200 2734 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x6
) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/142.0.0.0 Safari/537.36"
172.17.12.18 - - [27/Nov/2025:14:32:39 +0100] "GET /favicon.ico HTTP/1.1" 404 513 "https://172.17.23.90/" "Mozilla/5
0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/142.0.0.0 Safari/537.36"
172.17.12.18 - - [27/Nov/2025:14:32:56 +0100] "GET / HTTP/1.1" 200 2734 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x6
) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/142.0.0.0 Safari/537.36"
172.17.12.18 - - [27/Nov/2025:14:33:30 +0100] "-" 408 0 "-" "-"
172.17.12.18 - - [27/Nov/2025:14:33:52 +0100] "GET / HTTP/1.1" 200 2734 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x6
; rv:145.0) Gecko/20100101 Firefox/145.0"
172.17.12.18 - - [27/Nov/2025:14:33:52 +0100] "GET /favicon.ico HTTP/1.1" 404 513 "https://172.17.23.90/" "Mozilla/5
0 (Windows NT 10.0; Win64; x64; rv:145.0) Gecko/20100101 Firefox/145.0"
172.17.12.18 - - [27/Nov/2025:14:34:44 +0100] "GET / HTTP/1.1" 200 2734 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x6
) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/142.0.0.0 Safari/537.36"
172.17.12.18 - - [27/Nov/2025:14:34:47 +0100] "GET / HTTP/1.1" 200 981 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64
 AppleWebKit/537.36 (KHTML, like Gecko) Chrome/142.0.0.0 Safari/537.36"
root@debian-12:~#
```

---

# Partie 7 : Questions (Annexe)

1) Pourquoi utilise-t-on un certificat autosigné en développement, mais pas en production ?

On utilise un certificat autosigné en développement pour tester HTTPS sans coût, mais pas en production car il n'est pas reconnu par les navigateurs et n'assure pas la confiance ni la sécurité des utilisateurs.

2) Quelle commande permet de renouveler un certificat autosigné avant son expiration ?

```
openssl x509 -signkey key.pem -in cert.pem -days 365 -out
cert-renewed.pem
```

→ -days 365 : durée de validité (à ajuster)
→ key.pem : clé privée
→ cert.pem : certificat autosigné

3) Comment forcer la redirection HTTP → HTTPS dans Apache ? (à mettre en œuvre)

```
nano /etc/apache2/sites-available000-default.conf
```

```
<VirtualHost *:80>
    ServerName 172.17.23.90
    DocumentRoot /var/www/html

    # Redirection HTTP vers HTTPS
    Redirect "/" "https://172.17.23.90/"
```

```
</VirtualHost>
```

Il faudra ensuite recharger le service

```
sudo a2ensite site-secure.conf
sudo systemctl reload apache2
```

4) Quels sont les ports par défaut pour HTTP et HTTPS ? (à afficher à l'aide d'une commande)

Afin de voir les ports on peut faire la commande suivante :

```
grep -i 'listen' /etc/apache2/ports.conf
```

```
root@debian-12:~# grep -i 'listen' /etc/apache2/ports.conf
Listen 80
        Listen 443
        Listen 443
```

On a bien 80 pour HTTP et 443 pour HTTPS.

5) Comment vérifier la version de TLS utilisée ?

Afin de voir la version utilisée, on peut faire la commande suivante.

```
openssl s_client -connect 172.17.23.90:443
```

On cherchera la ligne "Protocol". Dans mon cas il s'agit de TLS version 1.3

```
Protocol  : TLSv1.3
```

⚠️Il faut privilégier TLS 1.2 ou 1.3 car les versions 1.0 et 1.1 des vulnérabilités sont connues.

6) Comment modifier la version de TLS dans Apache ?

```
nano /etc/apache2/sites-available/site-secure.conf
```

Il faudra ajouter la ligne suivante si on veut du TLS 1.3 uniquement.

```
SSLProtocol TLSv1.3
```

Nous ne le ferons pas car nous sommes déjà en version 1.3. A noter qu'il ne faut pas oublier de redémarrer le service.

7) Quelle commande permet de lister tous les chierements (ciphers) supportés par le serveur ?

```
openssl ciphers -v 'ALL:!ADH:!LOW:!EXP:!MD5:!3DES:!PSK:!SRP'
```

8) Quelles sont les suites de chiffrement recommandées par l'ANSSI ?

L'ANSSI recommande d'utiliser TLS 1.2 et 1.3 avec des suites modernes comme ECDHE + AES-GCM ou CHACHA20-POLY1305 et de désactiver les anciens algorithmes comme RC4, DES ou SHA1.

9) Renforcer la sécurité de votre configuration Apache en précisant les chierements autorisés (SSLCipherSuite).

Pour renforcer Apache, il faut forcer des suites de chiffrement sûres en ajoutant dans le VirtualHost HTTPS :

nano /etc/apache2/sites-available/site-secure.conf

Il faudra rajouter ceci :

```
SSLCipherSuite
ECDHE-ECDSA-AES256-GCM-SHA384:ECDHE-RSA-AES256-GCM-SHA384:ECDHE-ECDSA-AE
S128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256
SSLHonorCipherOrder on
SSLProtocol TLSv1.2 TLSv1.3
```

→ SSLCipherSuite → force uniquement les ciphers sûrs
→ SSLHonorCipherOrder on → applique la préférence du serveur
→ SSLProtocol → active seulement TLS 1.2 et TLS 1.3

Le fichier entier doit être comme ceci :

```
<IfModule mod_ssl.c>
<VirtualHost *:443>
ServerName 172.17.23.90
DocumentRoot /var/www/html
SSLEngine on
SSLCertificateFile /etc/apache2/ssl/server.crt
SSLCertificateKeyFile /etc/apache2/ssl/server.key
ErrorLog ${APACHE_LOG_DIR}/error.log
CustomLog ${APACHE_LOG_DIR}/access.log combined
SSLCipherSuite
ECDHE-ECDSA-AES256-GCM-SHA384:ECDHE-RSA-AES256-GCM-SHA384:ECDHE-ECDSA-AE
```

```
S128-GCM-SHA256:ECDHE-RSA-AES>
SSLHonorCipherOrder on
SSLProtocol TLSv1.2 TLSv1.3
</VirtualHost>
</IfModule>
```

Ensuite :

```
sudo apache2ctl configtest   # Vérifie la syntaxe
sudo systemctl reload apache2
```

Après cela, le serveur ne devrait plus accepter uniquement les ciphers sécurisés recommandés.

10) Tester.

```
openssl s_client -connect 172.17.23.90:443
```

On devrait retrouver

```
Protocol  : TLSv1.3
Cipher    : TLS_AES_128_GCM_SHA256
```

11) Conclusion (5 lignes)

Renforcer la sécurité TLS d'Apache est essentiel pour protéger les données et garantir la confiance des utilisateurs.
L'utilisation de TLS 1.2 et 1.3 avec des suites de chiffrement modernes comme ECDHE + AES-GCM ou CHACHA20-POLY1305 est recommandée.
Les certificats doivent être valides et signés par une autorité reconnue pour éviter les alertes et attaques man-in-the-middle.
Il est important de configurer Apache avec SSLCipherSuite, SSLProtocol et SSLHonorCipherOrder pour n'accepter que les ciphers sûrs.
Enfin, il faut toujours tester la configuration avec OpenSSL, Nmap ou des services en ligne pour s'assurer de sa robustesse et conformité aux bonnes pratiques.